

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Чукачев Григорий

Группа J3212

Проверил:

Добряков Д. И.

Санкт-Петербург, 2026 г.

Задача:

Мигрировать ранее разработанное веб-приложение на frontend-фреймворк Vue.JS. Приложение должно соответствовать следующим требованиям:

1. Являться одностраничным веб-приложением (SPA).
2. Иметь подключенный роутер для навигации.
3. Осуществлять работу с внешним API посредством библиотеки axios.
4. Демонстрировать компонентный подход.
5. Использовать механизм composables для вынесения повторяющегося функционала.

Цель работы:

Изучить основы фреймворка Vue.js, научиться разбивать приложение на переиспользуемые компоненты, управлять реактивным состоянием и настраивать клиентскую маршрутизацию.

Ход работы:

1. Инициализация проекта и настройка окружения:

С помощью инструмента create-vue (на базе сборщика Vite) был создан пустой проект. В качестве зависимостей были установлены vue-router для маршрутизации, axios для HTTP-запросов к API, а также bootstrap, bootstrap-icons и chart.js для сохранения прежнего визуального стиля и графиков. Файл базы данных db.json и сервер json-server были интегрированы прямо в проект для удобства запуска.

2. Настройка клиентской маршрутизации (Vue Router) :

Для обеспечения работы приложения в формате SPA был настроен роутер. В папке src/views были созданы страницы (Views): авторизации, регистрации, дашборда, операций, отчетов и настроек. В конфигурации роутера каждый путь был привязан к соответствующему компоненту-странице.

3. Декомпозиция интерфейса на компоненты:

Для соблюдения принципа DRY (Don't Repeat Yourself) глобальные и сложные элементы были выделены в отдельные компоненты (src/components):

- HeaderComponent.vue — навигационная панель, которая теперь подключается один раз в главном файле App.vue.
- На странице Дашборда сложная верстка была разбита на независимые виджеты: AccountsWidget.vue (счета), FinanceWidget.vue (рынки валют и акций), RecentTransactionsWidget.vue (список операций) и TransactionModal.vue (модальное окно).

4. Использование Composables:

Функционал переключения светлой и темной тем ("Зомби-мод"), а также сохранения выбранной темы в localStorage, был вынесен в отдельный composable-файл useTheme.js. Это позволило инкапсулировать реактивную логику управления темой и легко подключать ее в компоненте шапки сайта.

5. Работа с внешним API и реактивностью:

Запросы к локальному серверу (json-server) и стороннему API (курсы валют ЦБ РФ) были переписаны с использованием axios. Благодаря реактивности Vue, сложная логика фильтрации транзакций по типу, категории, сумме и дате была реализована через вычисляемые свойства (computed). Таблица обновляется мгновенно при изменении любого параметра (через двустороннее связывание v-model), что позволило отказаться от ручного перестроения DOM-дерева.

6. Интеграция графиков и связи между компонентами:

На странице отчетов был успешно интегрирован график Chart.js, данные для которого реактивно собираются из базы данных. Также была реализована связь между компонентами: компонент модального окна TransactionModal.vue после успешного добавления операции отправляет

событие родительскому компоненту (Дашборду), чтобы тот автоматически перезапросил актуальные данные с сервера.

Вывод:

В ходе выполнения работы приложение было успешно мигрировано на Vue.js. Задачи по настройке SPA, интеграции с API, использованию роутера, компонентов и composables были выполнены в полном объеме.

Сравнение с предыдущей версией показало, что использование фреймворка значительно упрощает разработку:

1. Двустороннее связывание (v-model) избавляет от необходимости писать шаблонный код для получения значений из форм.
2. Реактивность и вычисляемые свойства (computed) делают логику фильтрации и подсчета (например, лимитов или суммы расходов) прозрачной и надежной.
3. Компонентный подход позволяет сделать код более читаемым, поддерживаемым и легко масштабируемым.